REPORT

Reviewer report: mahmoud/boltons#746: Restore disabled

GC state on context exit

Run boltons-746, tested 2026-11-09 11:41 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 15:04 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: High

The PR's tests behave the same or better than on main, in all 3 runs per side. Every check passed and every

claim cites a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Restore disabled GC state on context exit (mahmoud/boltons#746).

• What we observed: 1 test(s) fail on main and pass on the PR; 5 test(s) ran on main, 5 on the PR; 0 new test(s)

pass, 4 unchanged; probe: claim(s) C1, C2, C3 proven.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 5.8 GB (of 15.6 GB).

• Setup check: free disk 92.5 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: boltons python setup install instructions https://github.com/mahmoud/boltons

site:github.com/mahmoud/boltons

• Query: boltons python documentation install https://github.com/mahmoud/boltons

site:github.com/mahmoud/boltons

• Query: boltons python install requirements https://pypi.org/project/boltons/ https://github.com/mahmoud/boltons

site:pypi.org

• Source: boltons · PyPI — https://pypi.org/project/boltons/

• Source: boltons/.github/workflows/tests.yaml at 4e5faa3d7e4008d89e0d8bf1ea87b6d9a061a16d · mahmoud/boltons · GitHub — https://github.com/mahmoud/boltons/blob/4e5faa3d7e4008d89e0d8bf1ea87b6d9a061a16d/.github/workflows/tests.yaml

• Source: GitHub - mahmoud/boltons at 4e5faa3d7e4008d89e0d8bf1ea87b6d9a061a16d · GitHub — https://github.com/mahmoud/boltons/tree/4e5faa3d7e4008d89e0d8bf1ea87b6d9a061a16d

• Source: boltons/.readthedocs.yaml at master · mahmoud/boltons · GitHub — https://github.com/mahmoud/boltons/blob/master/.readthedocs.yaml

• Source: boltons/docs/index.rst at master · mahmoud/boltons · GitHub — https://github.com/mahmoud/boltons/blob/master/docs/index.rst

• Source: boltons/boltons/ecoutils.py at 4b1d728f31a8378b193be9c966c853be0a57527d · mahmoud/boltons · GitHub — https://github.com/mahmoud/boltons/blob/4b1d728f31a8378b193be9c966c853be0a57527d/boltons/ecoutils.py

• Setup plan: the same packages on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.7

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that passes on main fails on the PR (4 unchanged, 3 runs per side) | pytest-pr-1.log:7 «5 passed in 0.67s»

| P-C1 | Restore the disabled GC state when collection was disabled at context entry. (PR description) | proven | probe model-p1, c1.gc_state_after_exception: value: true -> false | probe-model-p1-pr-1.jsonl:1 «"observation": "c1.gc_state_after_exception"»

| P-C2 | The enabled state already has a corresponding restoration path. (PR description) | proven | probe

model-p1, c2.gc_state_after_normal_exit: same on both sides: true | probe-model-p1-pr-1.jsonl:2 «"observation":

"c2.gc_state_after_normal_exit"»

| P-C3 | The new regression disables GC, enables it inside toggle_gc, then raises from the body. It fails on

d31d821 and passes with the change. (PR description) | proven | probe model-p1, c3.gc_state_after_exception:

value: true -> false | probe-model-p1-pr-1.jsonl:3 «"observation": "c3.gc_state_after_exception"»

Citation check (PASS): 4 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 5 / 5 | | Tests unchanged | 4 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 1 |

tests.test_gcutils::test_toggle_gc_restores_disabled_state_after_exception

| Tests that are new or changed but did not run because an optional module is missing | 0 | -

| Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 67734e1296af

at 2026-11-09T06:14:13.946368+00:00): Tests for claims C1, C2, C3

• P-C1 (PR description): Restore the disabled GC state when collection was disabled at context entry. Expects

the covered observations to differ.

• P-C2 (PR description): The enabled state already has a corresponding restoration path. Expects the covered

observations to stay the same.

• P-C3 (PR description): The new regression disables GC, enables it inside toggle_gc, then raises from the body.

It fails on d31d821 and passes with the change. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.gc_state_after_exception | P-C1 | as claimed | value: true -> false | probe-model-p1-main-1.jsonl:1

«"observation": "c1.gc_state_after_exception"»
probe-model-p1-pr-1.jsonl:1 «"observation":

"c1.gc_state_after_exception"»

| c2.gc_state_after_normal_exit | P-C2 | as claimed | same on both sides: true | probe-model-p1-main-1.jsonl:2

«"observation": "c2.gc_state_after_normal_exit"»
probe-model-p1-pr-1.jsonl:2 «"observation":

"c2.gc_state_after_normal_exit"»

| c3.gc_state_after_exception | P-C3 | as claimed | value: true -> false | probe-model-p1-main-1.jsonl:3

«"observation": "c3.gc_state_after_exception"»
probe-model-p1-pr-1.jsonl:3 «"observation":

"c3.gc_state_after_exception"»

Citation check (PASS): 6 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (2.1 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| boltons/gcutils.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 5520b6d4fd6b of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 4 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 6 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

Nothing went wrong.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 4 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 6 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-09/boltons-746 DECISION=approve (or reject, approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/mahmoud/boltons/pull/746

Evidence links

• Pull request mahmoud/boltons#746
• record.json
• pr.diff
• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-p1-main-1.jsonl
• probe-model-p1-pr-1.jsonl
• main run 1 result: pytest-main-1.log:7 «4 passed in 0.69s»
• pr run 1 result: pytest-pr-1.log:7 «5 passed in 0.67s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 15:04 IST